

3. Bölüm

Derleyiciler ve Çalıştırma Ortamı

3.1 Derleyici

Genel amaçlı bilgisayarların çok kullanılmasıyla birlikte metin editörleri de (Windows Not Defteri, Mac TextEdit, Notepad++, Epsilon, EMACS, nano, vim veya vi) işletim sistemlerinin bir parçası olmuştur. Metin editörlerinde yazılan programlar ise **derleyiciler (compiler)** tarafından makine koduna çevrilerek **icra edilebilir (executable)** dosya olarak kaydedilebilmektedir.

C++ dilinde programlama öğrenmeye başlamak için ilk adım, programı yazabileceğiniz bir metin editörü kullanmak ve yazdığınız programa ilişkin kaynak kodları **.cpp** uzantısı ile dosya olarak saklamaktır. İkinci adım ise **işletim sisteminizde (operating system)** çalışabilen bir icra edilebilir dosya oluşturan bir derleyici kurmaktır. Yani bilgisayarınızda iki yazılım aracına ihtiyacınız vardır; birincisi C++ derleyicisi, ikincisi ise basit bir metin editörüdür.

Derleyici için girdi, kaynak kodu dosyasıdır. Kaynak dosyasında yazılan kaynak kodu, içerisinde C++ **talimatları (statement)** olan, insan tarafından okunabilen metin dosyasıdır. Kaynak kodda verilen talimatlara göre işlemcinizin programı çalıştırabilmesi için, bunun makine diline **derlenmesi (compile)** gerekir.

Derleyici

Derleme, bir başka programlama dilinden **sembolik isim (mnemonic)** ya da **makine koduna (instruction code)** dönüştürme işlemidir. Bunu yapan programlara **derleyici (compiler)** adı verilir.

3.2 C++ derleyicileri

Günümüzde birçok C++ derleyicisi mevcuttur. En çok kullanılanları aşağıda verilmiştir;

- ♦ **GNU Derleyici Koleksiyonu (GCC):** Popüler bir açık kaynaklı C++ derleyicisidir¹. Windows, macOS ve Linux dahil olmak üzere çok çeşitli platformlarda kullanılabilir. GCC, çok çeşitli özellikleri ve çeşitli C++ standartlarına desteğiyle bilinir.
- ♦ **Clang:** LLVM projesinin bir parçası olan açık kaynaklı bir C++ derleyicisidir². Windows, macOS ve Linux dahil olmak üzere çok çeşitli platformlarda kullanılabilir. Clang, hızı ve optimizasyon yetenekleriyle bilinir.
- ♦ **Microsoft Visual C++:** Microsoft tarafından geliştirilen tescilli bir C++ derleyicisidir³. Yalnızca Windows için kullanılabilir. Visual C++, Microsoft Visual Studio geliştirme ortamıyla entegrasyonu ile bilinir.

Bilgisayarınızın 64 bitlik Windows olduğu varsayılarak **g++** derleyicisi aşağıdaki şekilde kurulur;

- ♦ MINGW sürümü ilgili web sayfasından indirilerek kurulur⁴.
- ♦ Kurulumu tamamlandıktan sonra **PATH** değişkenine **g++** derleyicisinin klasörü eklenir.

Böylece kurulum tamamlanmış olur. **Komut istemi (command prompt veya terminal)** açıldığında **g++ -v** komutu yazılarak derleyicinin düzgün kurulup kurulmadığı kontrol edilebilir.

```
C:\Users\ILHANOZKAN>g++ -v
Using built-in specs.
COLLECT_GCC=g++
COLLECT_LTO_WRAPPER=D:/msys64/ucrt64/bin/./lib/gcc/x86_64-w64-mingw32/13.2.0/lto-wrapper.exe
Target: x86_64-w64-mingw32
Configured with: ./gcc-13.2.0/configure --prefix=/ucrt64 --with-local-prefix=/ucrt64/local
--build=x86_64-w64-mingw32 --host=x86_64-w64-mingw32 --target=x86_64-w64-mingw32
--with-native-system-header-dir=/ucrt64/include --libexecdir=/ucrt64/lib --enable-bootstrap
--enable-checking=release --with-arch=nocona --with-tune=generic
```

¹<https://gcc.gnu.org/>

²<https://clang.llvm.org/>

³<https://visualstudio.microsoft.com/tr/vs/features/cplusplus/>

⁴<https://www.mingw-w64.org/downloads/>

```
--enable-languages=c,lto,c++,fortran,ada,objc,obj-c++,jit --enable-shared --enable-static
--enable-libatomic --enable-threads=posix --enable-graphite --enable-fully-dynamic-string
--enable-libstdcxx-filesystem-ts --enable-libstdcxx-time --disable-libstdcxx-pch
--enable-lto --enable-libgomp --disable-libssp --disable-multilib --disable-rpath
--disable-win32-registry --disable-nls --disable-werror --disable-symvers --with-libiconv
--with-system-zlib --with-gmp=/ucrt64 --with-mpfr=/ucrt64 --with-mpc=/ucrt64 --with-isl=/ucrt64
--with-pkgversion='Rev3, Built by MSYS2 project'
--with-bugurl=https://github.com/msys2/MINGW-packages/issues --with-gnu-as --with-gnu-ld
--disable-libstdcxx-debug --with-boot-ldflags=-static-libstdc++
--with-stage1-ldflags=-static-libstdc++
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 13.2.0 (Rev3, Built by MSYS2 project)
```

Kaynak kodumuzu yazmak için bir metin editörü gereklidir. **Windows** işletim sisteminde **notepad**, **Linux** ve **macOs** işletim sistemleri için **nano** programları metin yazmak için idealdir. İlk kaynak kodumuzu oluşturmak için Windows komut isteminde **notepad hello.cpp**, Linux ve macOS işletim sistemlerinde ise **nano hello.cpp** komutu yazılır. Editör uygulaması açılır. Eğer böyle bir dosya yoksa metin editörü bu dosyayı oluşturmak için onay ister. Açılan editörü penceresine ilk C++ programı yazılır ve **hello.cpp** adlı dosyaya kaydedilir.

```
#include <iostream>
int main() {
    std::cout << "Hello World!";
    return 0;
}
```

Kaydedilen dosyayı komut isteminde **g++ hello.cpp -o hello.exe** komutu ile derlenir. Böylece hazırlanan kaynak kodumuz derleyici tarafından derlenir (**compile**). Böylece kaynak kodumuz icra edilebilir (**executable**) **hello.exe** dosyasına dönüştürülmüş olur. Derleme ve çalıştırma sürecini **Windows** komut isteminde yapmak için aşağıdaki komutlar yazılır;

```
C:\Users\ILHANOZKAN>notepad hello.cpp
C:\Users\ILHANOZKAN>g++ hello.cpp -o hello.exe
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>
```

Benzer şekilde derleme ve çalıştırma sürecini **Linux** veya **macOs** komut isteminde yapmak için aşağıdaki komutlar yazılır;

```
ozkan@ILHANOZKAN:~$ nano hello.cpp
ozkan@ILHANOZKAN:~$ g++ hello.cpp -o hello.exe
ozkan@ILHANOZKAN:~$ ./hello.exe
Hello, World!
ozkan@ILHANOZKAN:~$
```

3.3 Entegre Geliştirme Ortamı

Şimdiye kadar anlatılan kod yazma (**implementation**), derleme (**compile**), icra (**execute**) ya da koşma (**run**) ve izleme (**trace**) gibi süreçlerin tamamını tek bir çatı altında yürütmemizi sağlayan programlar, entegre geliştirme ortamı kısaca **IDE (Integrated Development Environment)** olarak adlandırılır. IDE'ler kodumuzu renklendirir (**source code coloring** veya **syntax highlighting**) ve kod yazarken daha az hata yapmamızı sağlarlar. Ancak kodu dosyaya yine sade metin olarak kaydederler. Ayrıca kod yazarken talimatları **doğru tamamlamak (intellisense)** için programcıya yardımcı da olurlar. Bunun gibi kolaylaştırıcı birçok özellikleri bulunur. Örneğin yukarıda yazdığımız düz metin IDE içinde aşağıdaki gibi renkli görünür;

```
#include <iostream>
int main() {
    // İlk C++ Programı
    std::cout << "Hello World!";
    return 0;
}
```

C++ programlama dili için en çok kullanılan entegre geliştirme ortamları aşağıda verilmiştir;

- ◊ **Code::Blocks**: Açık kaynaklı bir C/C++ geliştirme ortamı olup bir grafik kullanıcı arayüzüne **GUI (Graphic User Interface)** sahiptir. Windows, macOS ve Linux işletim sistemleri üzerine kurulabilmektedir.
- ◊ **Visual Studio**: Microsoft tarafından geliştirilen, C++ dilinde yazılmış, güçlü, yüksek performanslı uygulamalar oluşturmak için kullanılabilen bir geliştirme ortamıdır. Visual Studio, **kod tamamlama (intellisense)**, kullanıcı arayüzü **UI (user interface)** hazırlama desteği ve **hata ayıklama (debugger)** ve birçok **eklentisi (plug-in)** olan muazzam özelliklere sahiptir.
- ◊ **Visual Studio Code**: Microsoft tarafından geliştirilen açık kaynaklı bir geliştirme ortamıdır. Windows, macOS ve Linux gibi işletim sistemlerinde çalışır. Git **sürüm kontrol (version control)** sistemleriyle çok iyi çalışır. Ayrıca, akıllı kod tamamlamanın dikkate değer özellikleriyle birlikte gelir.
- ◊ **CLion**: JetBrains tarafından geliştirilmiştir ve C++ programcıları için en çok önerilen çapraz platformlu (CMake derleme sistemiyle entegre macOS, Linux ve Windows altında çalışır) geliştirme ortamıdır. Ayrıca, yerel (**local**) ve uzaktan (**remote**) desteğe sahip birkaç IDE'den biri olarak kabul edilir. Bu da yerel bir makinede kod yazmanıza ancak uzak sunucularda derlemenize olanak tanır. Kaynak kodlarımız yönetmemizi sağlayan **CVS (Concurrent Versions System)** ve **TFS (Team Foundation Server)** ile entegre edilebilir.
- ◊ **Eclipse**: Java'da yazılmış ve IBM tarafından geliştirilmiş ücretsiz ve açık kaynaklı bir geliştirme ortamıdır. Yaklaşık otuz programlama dilini desteklediği için geniş topluluk desteğiyle iyi bilinir. C++ için Eclipse IDE, kod tamamlama, otomatik kaydetme, derleme ve hata ayıklama desteği, uzak sistem gezgini, statik kod analizi, **profilleme (profiling)** ve **yeniden düzenleme (refactoring)** gibi beklenebilecek tüm özelliklere sahiptir. Ayrıca çeşitli harici eklentileri entegre ederek işlevselliğini genişletebilirsiniz. Windows, Linux ve macOS'ta çalışabilir.
- ◊ **NetBeans**: Apache Yazılım Vakfı – Oracle Corporation tarafından geliştirilen ücretsiz ve açık kaynaklı bir geliştirme ortamıdır. Öğrenciler veya başlangıç seviyesindeki C/C++ geliştiricileri için şiddetle tavsiye edilmesinin sebebi, Eclipse'e benzer şekilde daha iyi sürükle ve bırak işlevlerine sahip olmasıdır. Windows, Linux, MacOSX ve Solaris gibi birden fazla platformda çalışır.
- ◊ **Xcode**: MacOSX işletim sisteminde yazılım geliştirmek için kullanılan bir geliştirme ortamıdır.

3.4 Çevrimiçi Derleyiciler

Bütün bu entegre geliştirme ortamlarının yanında online olarak da derleme yapılan web uygulamaları da bulunmaktadır. Bunlardan birkaçı aşağıda verilmiştir;

- ◊ https://www.tutorialspoint.com/compile_cpp_online.php
- ◊ https://www.onlinegdb.com/online_c++_compiler
- ◊ <https://www.programiz.com/cpp-programming/online-compiler/>
- ◊ <https://onecompiler.com/cpp>

3.5 Derleme Zamanı

Derleyici programının derleme işlemine başlayıp bitirildiği sürece **derleme zamanı (compile-time)** denir. Bu süreç başarısızlıkla da sonuçlanabilir ve eğer derleme işleminde hata meydana gelirse programcı hata mesajları ile uyarılır. Bir derleyici program, kaynak dosyayı makine diline çevirme çabasında, kaynak dosyanın C++ dilinin **sözdizimi (syntax)** kurallarına uygunluğunu da denetler.

Kod yazımı sırasında dilin kurallarına uyulmamışsa, derleyici bu durumu bildiren bir **derleme zamanı hatası (compile-time error)** alınarak derleme işlemi sonlandırılacaktır;

```

1 #include <iostream>
2 int main() {
3     std::cout << "Hello World!";
4     return
5 }
```

Örneğindeki gibi hatalı yazılmış bir kaynak kod aşağıdaki hatayı verecektir;

```

ozkan@ILHANOZKAN:~$ g++ hello.cpp -o hello.exe
hello.cpp: In function 'int main()':
hello.cpp:5:1: error: expected primary-expression before '}' token
5 | }
  | ^
hello.cpp:4:15: error: expected ';' before '}' token
4 |         return
  |         ^
  |         ;
```

```
5 | }
| ~
ozkan@ILHANOZKAN:~$
```

Kod yazımı sırasında dilin kurallarına uyulmuş ise bazı durumlarda **derleme zamanı uyarısı** (**compile-time warning**) alınabilir. Ancak derleme bu durumda tamamlanır.

Kaynak kod içerisindeki metinlerde Türkçe karakter kullanılması halinde **Windows** altında derlenen programı çalıştırdığınızda aynı metni göremeyebilirsiniz. Bunu düzeltmek için komut isteminde **Chcp 65001** komutu verilerek komut satırının UTF-16 karakter kümesi ile işlem yapmasını sağlayabiliriz.

3.6 Hatalar

Programcı tarafından kodlama yapılırken genellikle aşağıdaki üç tip hata yapılır;

Hatalar

1. **Derleme zamanı hataları** (**compile time error**): Bu tip hatalar genelde kullanılan dilin yazım kurallarına (**syntax**) uyulmadığından, talimatların (**statement**) yanlış yazılmasından ya da programcının kod metninde uygun olmayan karakterlerin kullanılmasından kaynaklanır.
2. **Koşma zamanı hataları** (**run time error**): Kaynak kod, kurallara uygun olarak yazılmıştır ve herhangi bir yazım hatası bulunmaz. Bu tip hatalara en iyi örneklerden birisi sıfıra bölme hatasıdır. Taşma hataları da bu hatalardandır. Daha sonra değinilecektir.
3. **Mantıksal hatalar** (**logical error**): Programcının çözüm için gerekli adımların oluşturulmasında, çözüm yönteminin yanlış olmasından ya da yanlış işlemler (**operator**) kullanılmasından kaynaklanır. Örneğin bir büyüktür (>) işleci yerine küçüktür (<) işleci kullanıldığında ne bir yazım hatası ne de bir çalışma zamanı hatası ortaya çıkar. Fakat program kendisinden istenilen davranışları yerine getirmez ve uygun çıktıları üretmez. Faktöriyel hesaplanırken $0!=1$ yerine $0!=0$ kabul etmek buna örnek verilebilir.

3.7 C++ Dilinin Kullanım Alanları

C dili, başlangıçta sistem geliştirme çalışmaları için, özellikle işletim sistemini oluşturan programlar için kullanıldı. Montaj (**assembly**) dilinde yazılan kod kadar hızlı çalışan kod ürettiği için sistem geliştirme dili olarak benimsendi. Bilgisayar mühendislerinin bilmesi gereken bu dilin birçok kullanım alanı mevcuttur. C++ ise;

- ♦ Çok kullanılan ve popüler programlama dillerinden biridir. C++ işletim sistemleri, gömülü sistemler ve Grafiksel Kullanıcı Arayüzleri yapımında kullanılır.
- ♦ Nesne yönelimli programlamayı destekler ve Soyutlama (**abstraction**), Sarma (**encapsulaton**) ve Miras alma (**inheritance**) gibi tüm OOP kavramlarını uygular, bu da programlara net bir yapı kazandırır ve kodun yeniden kullanılmasına olanak tanır, bu da geliştirme maliyetlerini düşürür ve güvenlik sağlar.
- ♦ Söz dizimi C++ diline benzediği için programcıların Java ve C# dillerine geçişini kolaylaştırır. C++ dili ara dil (**intermediate language**) kullanmaz ve kodu doğrudan işlemcinin emir kodlarına derler. Bu nedenle C#, Python ve Java'dan daha hızlıdır.
- ♦ C'nin de bir seçenek olduğu sınırlı kaynak ortamlarında kullanışlıdır. Ayrıca yürütme hızına ihtiyaç duyduğumuz veya donanım yakın çalışmamız gereken yerlerde de kullanılır.
- ♦ C ile karşılaştırıldığında C++, daha zengin kütüphanelere sahiptir, nesne yönelimli programlamayı destekler, şablonlar, istisna işleme ve daha birçok özelliği barındırır.

3.8 Düşün ve Çöz

- ♦ **Düşün:** "Derleme Zamanı Hatası" ile "Çalışma Zamanı Hatası" arasındaki farkı açıklayınız. Hangisinin bir mühendis için tespit edilmesi daha zordur? Neden?
- ♦ **Düşün:** Yorumlayıcı (**interpreter**) diller ile Derleyici (**compiler**) diller arasındaki performans farkının temel sebebi donanım seviyesinde nasıl açıklanır?
- ♦ **Düşün:** Statik bağlama (**static linking**) ve Dinamik bağlama (**dynamic linking**) arasındaki farkları araştırıp, programın dosya boyutu üzerindeki etkisini tartışınız.
- ♦ **Çöz:** Yazdığınız bir C kodunun makine diline dönüşüne kadar geçtiği aşamaları (Önişlemci →Derleyici →Bağlayıcı) sırasıyla listelleyiniz.
- ♦ **Çöz:** Bir C++ programının derleme aşamasında oluşan ara dosyaları (**.i**, **.s**, **.o**) ve bu dosyaların içeriklerini gösteren bir şema hazırlayınız.
- ♦ **Çöz:** Sadece bir metin editörü (notepad, nano vb.) ve bir derleyici (**g++** gibi) kullanarak "Merhaba Dünya" programını komut satırından derleyip çalıştırınız.

- ◇ **Düşün:** Derleme zamanı hatası (**compile-time error**), **bağlama zamanı hatası** (**link-time error**) ve çalışma zamanı hatası (**run-time error**) arasındaki farkı somut C++ kod örnekleriyle açıklayın.
- ◇ **Düşün:** Bir C++ programının kaynak koddan çalıştırılabilir dosyaya dönüşme sürecini adım adım açıklayın: önışlemci, derleyici, assembler ve bağlayıcı (**linker**) aşamalarını belirtin.
- ◇ **Düşün:** Çevrimiçi derleyiciler (Compiler Explorer, Godbolt vb.) ne gibi durumlarda kullanışlıdır? Yerel bir IDE'ye göre avantajları ve sınırlılıkları nelerdir?
- ◇ **Düşün:** C++ dilinin kullanıldığı üç farklı gerçek dünya uygulaması seçin (örn. Oyun motoru, işletim sistemi, gömülü sistem). Her birinde C++'ın tercih edilmesinin teknik gerekçesini açıklayın.
- ◇ **Düşün:** GCC ve Clang derleyicilerini karşılaştırın. Optimizasyon seviyeleri (**-O0**, **-O1**, **-O2**, **-O3**) oluşturulan makine kodunu nasıl etkiler?